An Architectural Blueprint for a Resilient and Transparent Video Upload Pipeline

This report presents an exhaustive diagnostic analysis and a principled engineering blueprint for the re-architecture of the application's core video loading and uploading functionality. The current implementation exhibits significant reliability and user experience deficits, including process stalls, silent failures, and a lack of transparent state reporting to the user. These issues are particularly detrimental to the target "Athlete" user persona, who requires a tool that is precise, dependable, and frictionless. The following analysis deconstructs the existing system's architectural flaws and proposes a robust, verifiable, and performant solution grounded in modern iOS development principles. The proposed architecture is designed to meet the highest engineering standards, ensuring long-term stability and maintainability for a large-scale user base.

Section 1: Categorical System Analysis of the Current Video Loading Pipeline

To systematically diagnose the root causes of the observed failures, the current video loading pipeline is first modeled using the formal language of category theory. This approach allows for a precise and unambiguous representation of the system's components, their interactions, and the emergent runtime behaviors that lead to instability.

1.1 Defining the Category of Video Ingestion

The system can be understood as a category where data structures and software modules are the **objects**, and the functions or data flows that transform them are the **morphisms**. This formalization provides a clear map of the system's structure.

Objects (Data Structures & Modules)

The key components of the current video ingestion pipeline are identified as the following

objects:

- **UserInteraction**: Represents the initial event triggered by the user, such as tapping a button to select a video from the device's photo library.
- **PHAsset**: An object from Apple's PhotosKit framework that represents the video asset selected by the user. It is a reference to the media, not the media data itself.
- **VideoFile**: Represents the concrete video data after it has been exported from the PHAsset. This object encapsulates the raw bytes of the video, residing temporarily on the device's local storage.
- **UploadRequest**: The network request object (e.g., a URLRequest instance) that is configured with the VideoFile data, server endpoint, and necessary HTTP headers for the upload.
- **ProgressState**: A data structure, likely a simple tuple or struct, that holds the current status of the loading process. In the current system, this appears to be a primitive representation, such as (isLoading: Bool, progress: Double), which is displayed to the user.
- **LoadingViewController**: The UI component responsible for rendering the ProgressState to the user, displaying the progress bar and status text (e.g., "Loading video from Photos...").

Morphisms (Functions & Data Flows)

The transformations and operations that connect these objects are defined as morphisms. These represent the active processes within the pipeline:

- selectVideo: UserInteraction ⟶ PHAsset: The function that presents the photo picker and returns the PHAsset chosen by the user.
- exportAsset: PHAsset ⟶ VideoFile: The asynchronous operation that requests the video data from PhotosKit and saves it to a temporary file, yielding a VideoFile object.
- createRequest: VideoFile ⟶ UploadRequest: A function that takes the local VideoFile and constructs a network UploadRequest.
- performUpload: UploadRequest ⟶ UploadResult: The core networking morphism that executes the upload and eventually returns a result, either success or failure.
- updateUI: ProgressState ⟶ LoadingViewController: The function that binds the current state data to the view, updating the progress bar and text labels.

1.2 Modeling Runtime Behavior as Morphism Composition

The entire user journey, from selecting a video to (ideally) completing the upload, can be modeled as a single, linear composition of these morphisms. This composition represents the

complete runtime behavior of the feature:

This model immediately reveals the primary architectural flaw of the system: its monolithic and brittle nature. The entire process is a single, tightly coupled chain of sequential operations. This structure is fundamentally ill-suited for the complexities of mobile application environments for several key reasons.

First, the model contains no alternative paths for error handling. A failure in any single morphism—for instance, if exportAsset fails due to insufficient disk space or performUpload times out due to a poor network connection—breaks the entire chain. The composition has no defined mechanism to catch this failure and branch to a recovery or error-reporting state. This explains the observed "silent failures"; the chain simply breaks, and the system is left in an indeterminate state with no further progression.

Second, the linear composition assumes a "happy path" where every operation succeeds promptly and in the correct order. This is an invalid assumption for any system involving file I/O and networking, which are inherently unpredictable. Mobile environments are characterized by fluctuating network conditions, resource constraints (CPU, memory), and user interruptions like backgrounding the app.[1] A robust architecture must be designed to anticipate and manage these conditions, not ignore them. The current model's failure to do so is a direct cause of its unreliability. The root cause is therefore not a bug within a specific function but a fragile architectural pattern—a single, non-resilient composition of morphisms that lacks the structure to handle the inevitable deviations from the ideal execution path.

Section 2: Root Cause Analysis via Morphism Chain Tracing

By mapping the observed failures—stalls, crashes, and UI discrepancies—back to the categorical model, it is possible to pinpoint the exact points where the morphism chain breaks and identify the underlying architectural deficiencies.

2.1 Tracing Failure Patterns in the Categorical Model

The provided logs and UI screenshots serve as empirical evidence of the system's runtime behavior, allowing for a detailed trace of the morphism chain's execution and failure modes.

Log and UI Discrepancy Analysis

A critical issue is the discrepancy between the UI state and the system's actual state. The UI often remains stuck on "Loading video from Photos..." even when, according to logs, the asset export is complete and the network upload has begun. This points to a fundamental flaw in how state updates are composed with system operations, which can be described as a **non-commutative diagram**.

In an ideal, correctly functioning system, the diagram of operations would commute. That is, the path of performing an operation and then updating the UI with its result should be equivalent to updating the UI to a "loading" state and then performing the operation. For the network phase, this would be:

This means the UI state should be a faithful reflection of the network operation's state at all times. However, the observed behavior reveals a non-commutative diagram. The updateUI morphism is invoked once at the beginning of the exportAsset phase, setting the text to "Loading video from Photos...". It is never composed again with the state changes that occur during the subsequent performUpload morphism. The state update and the actual operation do not commute because the UI update is decoupled from the true state of the system's progress. This leads directly to a misleading and untrustworthy user interface, which violates the core requirement for transparency.

2.2 Identifying Architectural Flaws

The non-commutative behavior is a symptom of deeper architectural flaws within the categorical model itself.

Missing Morphisms (Error Handling and State Transitions)

The model is critically incomplete because it lacks morphisms to handle failure states and intermediate transitions. For a robust system, the category must include morphisms such as:

- handleNetworkError: UploadRequestError \rightarrow ProgressState: A morphism to transform a network error into a user-facing state, such as "Connection Lost. Retrying...".
- retryUpload: UploadRequestError \rightarrow UploadRequest: A morphism that implements retry logic, potentially with exponential backoff, by re-initiating the upload.
- handleAppBackgrounded: AppState \rightarrow UploadState: A morphism to transition the upload process to a background-safe execution mode.

The absence of these morphisms means the system has no defined behavior for these common events, leading to unpredictable stalls and silent failures.

Buggy Isomorphism (State Representation)

The ProgressState object itself is flawed. The current implementation likely uses a simple data structure like a (Bool, Double) tuple to represent (isLoading, progress). While this is structurally isomorphic to a primitive state representation, it is a **buggy isomorphism** because it erases critical contextual information. For example, the distinct conceptual states of "Preparing video" and "Uploading video" might both be represented by the data (true, 0.1), making them indistinguishable to the UI layer. This loss of information is a direct cause of the opaque and unhelpful loading messages. A proper state representation would not be isomorphic to a simple tuple but would instead be a richer data type, such as an enum with associated values, capable of capturing the full context of each stage in the pipeline.

The Silent Killer: The Hidden Dependency on App Lifecycle

The most significant and damaging flaw is a hidden dependency that is not represented in the model at all: the application's lifecycle state (.active, .background, .suspended). The user's comment about needing to be "patient enough" strongly implies that they are backgrounding the app during the upload process, which then fails.

iOS imposes strict time limits on general-purpose background execution. When an app moves to the background, it is typically granted only a very short window of time—often around 30 seconds, and sometimes less—to complete any in-flight tasks before being suspended.[3] The current monolithic

performUpload morphism is not designed to operate within this constraint. When the user backgrounds the app, the operating system will eventually signal the process to suspend. Because the categorical model lacks a handleAppBackgrounded morphism to transition the work to a durable background execution API, the upload process is unceremoniously terminated by the OS. This results in the quintessential silent failure: the upload stops, the UI remains frozen in its last state, and the user is given no indication of what went wrong. The entire system is built on the implicit and fundamentally incorrect assumption that it will always operate in the foreground. This is the single greatest source of unreliability in the current architecture.

Section 3: A Principled Model for a Resilient Loading Experience

To rectify these deep-seated architectural issues, it is necessary to define a new, ideal system from first principles. This model will serve as the blueprint for the re-engineered solution,

ensuring that resilience, transparency, and correctness are built into its very structure.

3.1 The Ideal Category: A Formal Model of a Robust System

The ideal category for video ingestion is composed of more sophisticated objects and a richer set of morphisms that explicitly account for the complexities of the mobile environment.

New Objects

- **UploadJob**: A persistable data object that represents the entire unit of work. It encapsulates a unique identifier, the original PHAsset ID, the URL of the local temporary VideoFile, and metadata about the upload progress (e.g., which chunks have been successfully uploaded). Its persistence is key to enabling resumability across app launches.
- **UploadStateManager**: A dedicated manager component, ideally implemented as a Swift Actor to guarantee thread-safe state mutations. This object serves as the single source of truth for the state of all ongoing and pending UploadJobs. It is responsible for processing events and transitioning the system between states.
- **UploadState**: A rich enum with associated values that precisely models every possible stage of the job lifecycle. This object replaces the ambiguous (Bool, Double) tuple. Its cases would include:
  - idle
  - preparing(progress: Double)
  - compressing(progress: Double)
  - waitingForNetwork
  - uploading(chunk: Int, of: Int, progress: Double)
  - paused(reason: PauseReason) where PauseReason could be .noConnection, .userPaused, etc.
  - succeeded
  - failed(error: Error)
- **NetworkConditionMonitor**: An object, likely using Apple's NWPathMonitor, that observes the device's network reachability and interface type (e.g., Wi-Fi, Cellular). This provides crucial input for state transitions and adaptive compression logic.
- **BackgroundTaskManager**: An abstraction layer over the underlying iOS background task APIs (e.g., BGContinuedProcessingTask). This object is responsible for requesting, managing, and correctly terminating background execution time to ensure the upload can continue reliably when the app is not in the foreground.

New Morphisms

- startJob: PHAsset \rightarrow UploadJob: Creates and persists a new UploadJob

instance from a selected asset.
- transitionState: (UploadJob, Event) \rightarrow UploadState: The core logic of the state machine, which takes the current job and an incoming event (e.g., networkDidBecomeUnavailable) and produces the new UploadState.
- persistState: UploadState \rightarrow Disk: A morphism that saves the current state of an UploadJob to persistent storage, enabling recovery.
- resumeJob: Disk \rightarrow UploadJob: A morphism that rehydrates an UploadJob from storage upon app launch, allowing pending uploads to be resumed.

3.2 Constructing a Functor from the Ideal to the Observed System

A **functor** is a structure-preserving map between categories. To formally articulate the shortcomings of the current system, one can define a "simplification" or "forgetful" functor, , that maps the ideal, robust category defined above to the current, flawed one.

- maps the rich UploadState enum to the primitive (Bool, Double) tuple, demonstrating the loss of contextual information.
- maps the centralized, thread-safe state manager to a simple, transient property on the LoadingViewController, explaining the violation of SRP and the loss of state when the UI is dismissed.
- maps the complex, multi-step, resumable chunked upload process to the single, monolithic performUpload morphism, highlighting the loss of resilience.

This functorial mapping provides a rigorous way to describe what has been "forgotten" or omitted by the current implementation. It formally demonstrates that the existing system is a simplified, and therefore fragile, projection of what a truly robust system requires. The failures observed in production are the direct consequence of the information loss and structural simplification embodied by this functor.

3.3 Natural Transformations as Architectural Fixes

A **natural transformation** provides a systematic way to convert one functor into another. The proposed architectural fix can be modeled as a natural transformation, , from our forgetful functor to the identity functor on the ideal category (i.e.,). This transformation represents the set of component-wise functions needed to "upgrade" the flawed system into the ideal one.

For example, the component of the natural transformation at the UploadState object, , is the

function that replaces the simple (Bool, Double) tuple with the rich UploadState enum, thereby restoring the lost contextual information.

The most critical component of this transformation is the introduction of a **formal state machine**. The current system's state logic is implicit, scattered, and prone to race conditions. A state machine, as implemented in libraries like Machinus or through custom-built solutions, centralizes all state logic.[6] It provides a deterministic framework by defining:

1. A finite set of all possible states (our UploadState enum).
2. The specific, valid transitions that can occur between these states.
3. The actions or side effects that are executed upon entering or exiting a state.

Implementing a state machine directly addresses the UI transparency problem by creating a granular and accurate source of truth that the UI can simply observe. It also solves the problem of handling complex event sequences. For example, receiving a "network lost" event while in the uploading state can now deterministically and safely transition the system to the paused(reason:.noConnection) state. Therefore, the implementation of a state machine is the natural transformation that corrects the flawed state management functor, making the system's behavior predictable, verifiable, and robust.

Section 4: Categorization of Architectural Deficiencies

A systematic categorization of the current system's flaws reveals gaps in its components, unhandled events, and suboptimal processes. These deficiencies must be addressed to build a reliable pipeline.

4.1 Object Gaps (Missing Components)

The architecture is missing several critical objects required for a modern, resilient upload feature.

- **UploadStateManager**: There is no single, authoritative source for the state of an upload. State is likely managed directly within the LoadingViewController, which violates the Single Responsibility Principle (SRP). This tight coupling means that if the view controller is deallocated (e.g., the user navigates away), the entire state of the upload is lost, making recovery impossible.
- **ResumableUploader**: The system lacks a dedicated component for managing chunked uploads. Robust SDKs, such as those from Mux and Cloudinary, are built around this

concept, which is essential for handling large files and unreliable networks.[8] Without this object, the system is forced to use a monolithic upload strategy that is prone to failure.
- **BackgroundTaskManager**: There is no component responsible for interfacing with iOS's background execution frameworks. This is a critical omission, as managing the lifecycle of long-running background tasks is non-trivial and essential for ensuring that an upload can survive the app being backgrounded by the user.[10]

4.2 Missing Morphisms (Unhandled Events and Errors)

The system fails to define behaviors for numerous common and predictable events, resulting in an incomplete and fragile process.

- **handleAppBackgrounded**: No logic exists to transition the upload process to a background-safe execution context when the app's state changes.
- **handleConnectionLost**: The system does not react to changes in network connectivity. It cannot pause the upload when the connection is lost and resume it upon recovery.
- **handleUploadChunkFailure**: There is no retry logic for transient network errors that may cause a single chunk to fail. Standard practice dictates implementing a retry mechanism, often with exponential backoff, to handle such cases.[9]
- **handleAssetAccessDenied**: The system does not account for the edge case where a user revokes the app's access to the Photos library while an export or upload is in progress. This would lead to an unhandled crash or stall.

4.3 Inefficient Functors (Suboptimal Flows)

The existing processes, or functors, that map inputs to outputs are implemented in a suboptimal manner, leading to poor performance and a fragile user experience.

- **Foreground-Bound Video Processing**: The exportAsset morphism, along with any potential video compression, likely executes on the main thread or a simple global dispatch queue. This can block the UI, leading to a frozen interface, and makes the process highly susceptible to termination if the app is backgrounded.
- **Monolithic Upload**: Attempting to upload a large video file in a single HTTP request is fundamentally unreliable on mobile networks. It has a high probability of failure due to timeouts and is impossible to resume if interrupted.[8]
- **Absence of an "Adaptive Upload" Strategy**: The system employs a one-size-fits-all approach to video processing and uploading, failing to adapt to the user's current

context. The "Athlete" persona may be on a high-speed Wi-Fi network at home or a weak, congested cellular network at a remote training venue. Modern iPhones record video in highly efficient but potentially large formats like HEVC (H.265) or even professional-grade ProRes.[13] The principle of Adaptive Bitrate (ABR) streaming, where multiple quality renditions of a video are created to match a user's download bandwidth, provides a powerful analogue for uploading.[14] The current, inefficient functor always performs the same (or no) compression. A more efficient, context-aware functor would take both the VideoFile and the current NetworkCondition as input. On a fast Wi-Fi connection, it might upload the original file with minimal processing. On a slow cellular connection, it could perform a more aggressive transcode to a lower-bitrate H.264 format, significantly reducing the file size and upload time at the expense of on-device CPU cycles. This adaptive strategy is a crucial optimization for delivering a consistently reliable experience to the target user.

Section 5: Solution Architecture and Strategic Recommendations

The proposed solution is a complete re-architecture of the video upload pipeline, centered on a state-driven, background-first design that prioritizes resilience, transparency, and performance.

5.1 Core Strategy: A State-Driven, Background-First Architecture

The cornerstone of the new architecture will be a formal, actor-based UploadManager. This component will encapsulate a state machine governed by the rich UploadState enum defined previously. Using a Swift actor ensures that all state mutations are thread-safe and serialized, eliminating race conditions and providing a stable foundation for the entire system. This approach aligns with best practices for modern Swift concurrency and state management.[6]

The UploadManager will serve as the single source of truth, completely decoupled from the UI. The LoadingViewController or any other relevant UI component will simply subscribe to state updates published by the manager and render them. This separation of concerns ensures that the upload lifecycle is independent of the UI lifecycle, allowing uploads to persist even if the user navigates away from the loading screen.

5.2 Resilient Networking with Chunked, Resumable Uploads

To overcome the fragility of monolithic uploads, the new architecture will adopt a chunked uploading strategy. This is the industry standard for reliable large file transfer on mobile platforms.[8] The video file will be split into smaller, manageable chunks (e.g., 5-10 MB). Each chunk will be uploaded as an independent network request.

This approach provides several key benefits:

- **Resilience**: If the upload of a single chunk fails due to a transient network error, only that chunk needs to be retried, not the entire file. An exponential backoff retry mechanism will be implemented to handle such failures gracefully.
- **Resumability**: The state of each chunk's upload (e.g., pending, inProgress, completed) will be persisted to disk. This allows the UploadManager to resume an interrupted upload from the exact point it left off, even if the app is terminated and relaunched.[9]
- **Accurate Progress**: Progress can be reported with much higher fidelity, as it can be calculated based on the number of successfully uploaded chunks.

5.3 Selecting the Appropriate Background Task API

The choice of iOS background execution API is the most critical architectural decision for ensuring the upload process can survive the app being backgrounded. A comparison of the available APIs reveals the optimal choice for this specific user-initiated, long-running task.

Table 1: Comparison of iOS Background Task APIs

API	Initiation	Execution Time	System Discretion	Use Case	Suitability for Video Upload
beginBackgroundTask	Foreground	Short (approx. 30s)	High	Finishing short, critical tasks before	**Unsuitable**. The time allowance is far too short for

				suspension.	even moderately sized video uploads.[5]
BGProcessingTask	System-scheduled	Minutes	Very High	Deferred, non-urgent maintenance (e.g., database cleanup, ML model training) during periods of low device activity.	**Unsuitable.** The task is not user-initiated, and its execution timing is unpredictable and controlled by the system.[1]
Background URLSession	Foreground/Background	Indefinite (OS-managed)	Medium	Offloading long-running, non-discretionary network transfers entirely to the operating system.	**Viable, but with trade-offs.** This is an excellent choice for pure network transfers, as the OS handles the process even if the app is terminated. However, it offers less fine-grained control over the surrounding logic, such as pre-proces

					sing or chunk management, which must be completed before handing off to the session.[19]
BGContinuedProcessingTask	User-initiated (Foreground)	Minutes or more	Low	User-initiated, long-running jobs that must be allowed to continue if the app is backgrounded. Examples include video exporting and applying filters.	**Highly Suitable.** This API is explicitly designed for this use case. It allows the app to continue running for an extended period to perform both CPU-intensive work (like video compression) and networking. It provides the necessary control over the entire job lifecycle.[10]

Recommendation:

A hybrid approach leveraging BGContinuedProcessingTask as the primary lifecycle manager is recommended. When the user initiates an upload, the app will request a BGContinuedProcessingTask. This task provides a resilient execution window that persists even if the app is backgrounded. Within this protected window, the UploadManager will perform all necessary operations: video pre-processing (the adaptive compression), file chunking, and uploading the chunks using a standard URLSession. This architecture provides the ideal balance of control over the entire process and resilience against user interruptions and app lifecycle events.

5.4 Adjoint Functors for Optimization: The "Adaptive Upload" Hypothesis

To formalize the "Adaptive Upload" strategy, the concept of an **adjoint functor** from category theory can be used as a powerful model for optimization. This provides a principled basis for making intelligent trade-offs between on-device processing and network transfer time.

Consider two functors:

1. A functor that maps a given video and network condition to the total time required for the upload: .
2. A functor that represents the pre-processing step, mapping the video and network condition to a processed (e.g., compressed) video file: .

The goal is to find a pre-processing strategy that is a *left adjoint* to the upload time functor . In practical terms, this means finding the "most efficient" pre-processing step that minimizes the overall upload time . This formalizes the trade-off: on a slow network, the optimal solution is to spend more time executing (i.e., performing heavy on-device compression) to drastically reduce the size of the ProcessedVideo, which in turn minimizes the UploadTime. On a fast network, the optimal might be the identity function (i.e., do no compression) to avoid unnecessary CPU work and begin the network transfer immediately. This provides a robust theoretical foundation for the adaptive upload feature, ensuring it makes optimal decisions based on the user's real-time context.

Section 6: Phased Implementation and Verification Plan

The implementation will proceed in a structured manner, focusing on building and verifying each component of the new architecture. This plan ensures a methodical transition to the robust system, with clear deliverables and testing strategies at each phase.

6.1 Component Implementation with Targeted Code

The implementation will begin with the foundational state management components, followed by the background task integration and networking layer.

UploadState Enum and State Machine

The core of the UploadManager is its state machine. The states and transitions will be explicitly defined to ensure deterministic behavior.

Table 2: Video Upload State Machine Definition

Current State	Event	Action(s)	Next State	UI String
Idle	userSelectedVideo(asset)	Create UploadJob, request BGContinuedProcessingTask, start pre-processing.	Preparing(0.0)	"Preparing video..."
Preparing(progress)	preprocessingComplete(fileURL)	Initiate chunking logic, create URLSessionUploadTask for first chunk.	Uploading(chunk: 1,...)	"Uploading..."
Preparing(progress)	preprocessingFailed(error)	Log error, clean up, end background task.	Failed(error)	"Upload Failed"

Uploading(...)	networkConnectionLost	Pause URLSessionTask, persist state.	Paused(reason:.noConnection)	"Paused - No connection"
Uploading(...)	appWillEnterBackground	(No state change, execution is protected by BGContinuedProcessingTask).	Uploading(...)	"Uploading..."
Uploading(...)	uploadChunkSucceeded	Update progress, start upload for next chunk. If last chunk, transition to Succeeded.	Uploading(chunk: n+1,...) or Succeeded	"Uploading..." or "Upload Complete"
Uploading(...)	uploadChunkFailed(error)	Initiate retry logic. If max retries reached, transition to Failed.	Uploading(...) or Failed(error)	"Uploading..." or "Upload Failed"
Paused(...)	networkConnectionRestored	Resume URLSessionTask for the pending chunk.	Uploading(...)	"Uploading..."
* (Any State)	userCancelled	Cancel all tasks, clean up temp files, end background task.	Idle	""
* (Any State)	backgroundTa	Pause upload,	Paused(reason	"Paused"

	skExpired	persist state, prepare for graceful termination.	:.backgroundTaskExpired)	

UploadManager Actor Implementation

The UploadManager will be implemented as a Swift actor to ensure thread-safe access to its state.

Swift

```
import Foundation
import Photos

actor UploadManager {
    // Published properties for UI observation
    @Published private(set) var state: UploadState =.idle

    private var currentJob: UploadJob?
    //... other properties for network client, background task manager, etc.

    func startUpload(for asset: PHAsset) {
        // 1. Transition to Preparing state
        // 2. Create and persist UploadJob
        // 3. Request and begin BGContinuedProcessingTask
        // 4. Start asset export and adaptive compression
        //...
    }

    // Functions to handle events from network layer, app lifecycle, etc.
    func handleChunkSuccess(chunkIndex: Int) {
        //... transition state, start next chunk
    }

    func handleNetworkStatusChange(isConnected: Bool) {
        //... transition to Paused or resume from Paused
    }
```

```
    //... other event handling methods
}
```

BGContinuedProcessingTask Registration and Submission

The background task must be registered with the system at app launch and submitted when a user-initiated task begins.

Swift

```swift
// In your App's main struct or AppDelegate
import BackgroundTasks

private func registerBackgroundTask() {
    BGTaskScheduler.shared.register(
        forTaskWithIdentifier: "com.yourapp.videoUpload",
        using: nil
    ) { task in
        // This closure is called when the system launches the app in the background
        // to run the task. We need to hand off control to our UploadManager.
        if let task = task as? BGContinuedProcessingTask {
            // handleBackgroundTask(task: task)
        }
    }
}

// When the user starts an upload
private func submitBackgroundTask() {
    let request = BGContinuedProcessingTaskRequest(identifier: "com.yourapp.videoUpload")
    request.title = "Uploading Video" // For system UI
    request.subtitle = "Your video upload is in progress."

    do {
        try BGTaskScheduler.shared.submit(request)
    } catch {
        print("Could not schedule background task: \(error)")
    }
}
```

This setup ensures that the system is aware of the long-running task and will grant the app

the necessary resources to continue execution if it is backgrounded.[10]

6.2 Verifying Commutativity: Ensuring Architectural Correctness

The new state machine-based architecture inherently enforces correctness and eliminates the UI discrepancies seen in the old system. In this model, any event that can alter the system's state—a network response, a change in connectivity, an app lifecycle notification—*must* be routed through the UploadManager actor. The manager processes the event, deterministically transitions to a new state, and then publishes this new state to any observers (like the UI).

This enforces a **commutative diagram** by design:

The UI update becomes a pure, deterministic function of the state published by the UploadManager. There is no opportunity for the UI to fall out of sync, as it has no independent state management logic. This architectural pattern guarantees that what the user sees is a faithful and timely representation of the system's true state, fulfilling the core requirement for transparency.

6.3 A Robust Testing Strategy

A comprehensive testing suite is essential to verify the correctness and resilience of the new pipeline. Modern testing practices using Swift's async/await and XCTest will be employed.[22]

- **Unit Tests**: The UploadManager's state machine logic will be tested in isolation. Mock events (e.g., mockNetwork.triggerFailure()) will be sent to the actor, and its resulting state property will be asserted against the expected outcome. This verifies the correctness of every state transition defined in the table.
- **Integration Tests**: The full pipeline will be tested by creating mock implementations of the external dependencies, such as the networking layer (NetworkClient) and the PhotosKit interface. These tests will verify the correct interaction between the UploadManager, the uploader component, and the background task manager.
- **End-to-End (UI) Tests**: Automated UI tests will simulate user interactions, such as selecting a video, backgrounding the app, disabling the network, and then re-enabling it, to confirm that the UI correctly reflects the underlying state changes and that the upload successfully recovers and completes.

Example Asynchronous Test Case:

Swift

```swift
import XCTest

@testable import YourApp

final class UploadManagerTests: XCTestCase {

    // Test that the manager correctly pauses on network loss and resumes on recovery
    @MainActor
    func testUploadPausesAndResumesOnNetworkLoss() async throws {
        let mockNetworkClient = MockNetworkClient()
        let uploadManager = UploadManager(networkClient: mockNetworkClient)

        // Expectation for the final success state
        let successExpectation = expectation(description: "Upload should complete successfully after network recovery")

        // Subscribe to state changes to fulfill expectations
        let cancellable = uploadManager.$state.sink { state in
            if case.succeeded = state {
                successExpectation.fulfill()
            }
        }

        // 1. Start the upload
        await uploadManager.startUpload(for: MockPHAsset())

        // 2. Wait for the first chunk to start, then simulate network loss
        // (MockNetworkClient can be configured to do this)
        mockNetworkClient.setShouldFailAfterFirstChunk(true)

        // 3. Await the state becoming Paused
        await self.waitForState(.paused(reason:.noConnection), in: uploadManager)

        // 4. Simulate network recovery
        mockNetworkClient.setShouldSucceed(true)
```

```
        await uploadManager.handleNetworkStatusChange(isConnected: true)

        // 5. Await the final success state
        await fulfillment(of: [successExpectation], timeout: 30.0)

        cancellable.cancel()
    }

    // Helper to await a specific state change
    private func waitForState(_ targetState: UploadState, in manager: UploadManager) async {
        //... implementation using async stream or Combine to wait for the state to match
    }
}
```

6.4 Identifying and Mitigating Blind Spots

Even with a robust architecture, several potential blind spots must be proactively addressed to ensure production readiness.

- **Exotic Video Codecs (Apple ProRes)**: Newer iPhone models can record in Apple ProRes format, which produces exceptionally large files (up to 30 times larger than HEVC).[13] The "Adaptive Upload" pre-processing step must be able to detect ProRes files and make an intelligent decision. This may involve always forcing a transcode to a more web-friendly format like HEVC or H.264 and clearly communicating this processing step to the user to manage expectations about the initial "Preparing" phase.
- **Background Task Time Limits**: While BGContinuedProcessingTask provides an extended runtime, it is not infinite. The system can still terminate the task under conditions of extreme resource pressure. Therefore, the implementation must be fully resilient to this possibility. This means robustly saving the UploadJob state to disk *after every single successful chunk upload*. If the task is terminated, the UploadManager can resume the job from the last completed chunk on the next app launch.
- **Photos Library Permissions**: A user can change app permissions at any time via the Settings app. The code that accesses the PHAsset must handle potential PhotosError.accessRestricted or .accessUserDenied errors gracefully at every stage, not just at the initial selection. If permissions are revoked mid-process, the state machine must transition to a Failed(error:.permissionDenied) state and clean up resources.
- **Device Storage Space**: The pre-processing and chunking steps require temporary local storage. Before beginning the export from PhotosKit, the system must check for sufficient available disk space. If space is low, it should fail gracefully with an informative error message rather than crashing or filling up the user's device.

- **Server-Side Dependencies and Security**: This entire client-side architecture relies on the server supporting chunked, resumable uploads (e.g., via PUT requests with Content-Range headers or a dedicated multi-part upload API). This dependency must be verified with the backend team. Furthermore, for security, the app should not contain any long-lived API keys. The upload process should begin by requesting a short-lived, signed upload URL from the backend for each new UploadJob. This is a critical security best practice that prevents abuse of the upload endpoint.[8]

Works cited

1. Finish tasks in the background - WWDC25 - Videos - Apple Developer, accessed October 5, 2025, https://developer.apple.com/videos/play/wwdc2025/227/
2. Advances in App Background Execution - WWDC19 - Videos - Apple Developer, accessed October 5, 2025, https://developer.apple.com/videos/play/wwdc2019/707/
3. ios - How long does Apple permit a background task to run? - Stack Overflow, accessed October 5, 2025, https://stackoverflow.com/questions/28275415/how-long-does-apple-permit-a-background-task-to-run
4. Is it possible to run background task longer than 30 seconds? : r/iOSProgramming - Reddit, accessed October 5, 2025, https://www.reddit.com/r/iOSProgramming/comments/wxilem/is_it_possible_to_run_background_task_longer_than/
5. iOS: Perform upload task while app is in background - Stack Overflow, accessed October 5, 2025, https://stackoverflow.com/questions/23829546/ios-perform-upload-task-while-app-is-in-background
6. drekka/Machinus: Swift based state machine with many ... - GitHub, accessed October 5, 2025, https://github.com/drekka/Machinus
7. Even Better State Machines in Swift | by Markus Kasperczyk, accessed October 5, 2025, https://betterprogramming.pub/even-better-state-machines-3acd447652c6
8. iOS image and video upload | Documentation - Cloudinary, accessed October 5, 2025, https://cloudinary.com/documentation/ios_image_and_video_upload
9. muxinc/swift-upload-sdk: Mux's Video Upload SDK for iOS ... - GitHub, accessed October 5, 2025, https://github.com/muxinc/swift-upload-sdk
10. Performing long-running tasks on iOS and iPadOS | Apple Developer Documentation, accessed October 5, 2025, https://developer.apple.com/documentation/BackgroundTasks/performing-long-running-tasks-on-ios-and-ipados
11. Using background tasks to update your app | Apple Developer Documentation, accessed October 5, 2025, https://developer.apple.com/documentation/uikit/using-background-tasks-to-update-your-app
12. How do you get clients to upload iPhone videos without it taking HOURS (or

constantly crashing)? : r/socialmedia - Reddit, accessed October 5, 2025, https://www.reddit.com/r/socialmedia/comments/1mohgem/how_do_you_get_clients_to_upload_iphone_videos/
13. iPhone Video Formats - How to Convert Video Formats for iPhone, accessed October 5, 2025, https://www.videoproc.com/iphone-video-processing/iphone-supported-video-formats.htm
14. Using adaptive bitrate streaming for video optimization - Abraia, accessed October 5, 2025, https://abraia.me/docs/adaptive-bitrate/
15. Adaptive Bitrate Streaming: What it Is and How ABR Works - Dacast, accessed October 5, 2025, https://www.dacast.com/blog/adaptive-bitrate-streaming/
16. How I added resumable video uploads to My iOS App | by Niko - Medium, accessed October 5, 2025, https://nikodev1.medium.com/how-i-added-resumable-video-uploads-to-my-ios-app-ac60c058a942?source=rss------videos-5
17. Choosing Background Strategies for Your App | Apple Developer Documentation, accessed October 5, 2025, https://developer.apple.com/documentation/backgroundtasks/choosing-background-strategies-for-your-app
18. iOS BackgroundTasks - Amit Thakur, accessed October 5, 2025, https://amitkthakur.medium.com/ios-backgroundtasks-435a8e58b9e0
19. Downloading files in the background | Apple Developer Documentation, accessed October 5, 2025, https://developer.apple.com/documentation/Foundation/downloading-files-in-the-background
20. Background upload in iOS - by Diana Nareiko - Medium, accessed October 5, 2025, https://medium.com/@diananareiko8/background-upload-in-ios-f885ed439bd3
21. Uploading data in the background in iOS | Livefront Digital Product Consultancy, accessed October 5, 2025, https://livefront.com/writing/uploading-data-in-the-background-in-ios/
22. Unit testing Swift code that uses async/await | Swift by Sundell, accessed October 5, 2025, https://www.swiftbysundell.com/articles/unit-testing-code-that-uses-async-await/
23. Unit testing async/await Swift code - SwiftLee, accessed October 5, 2025, https://www.avanderlee.com/concurrency/unit-testing-async-await/
24. Asynchronous Tests and Expectations | Apple Developer Documentation, accessed October 5, 2025, https://developer.apple.com/documentation/xctest/asynchronous-tests-and-expectations